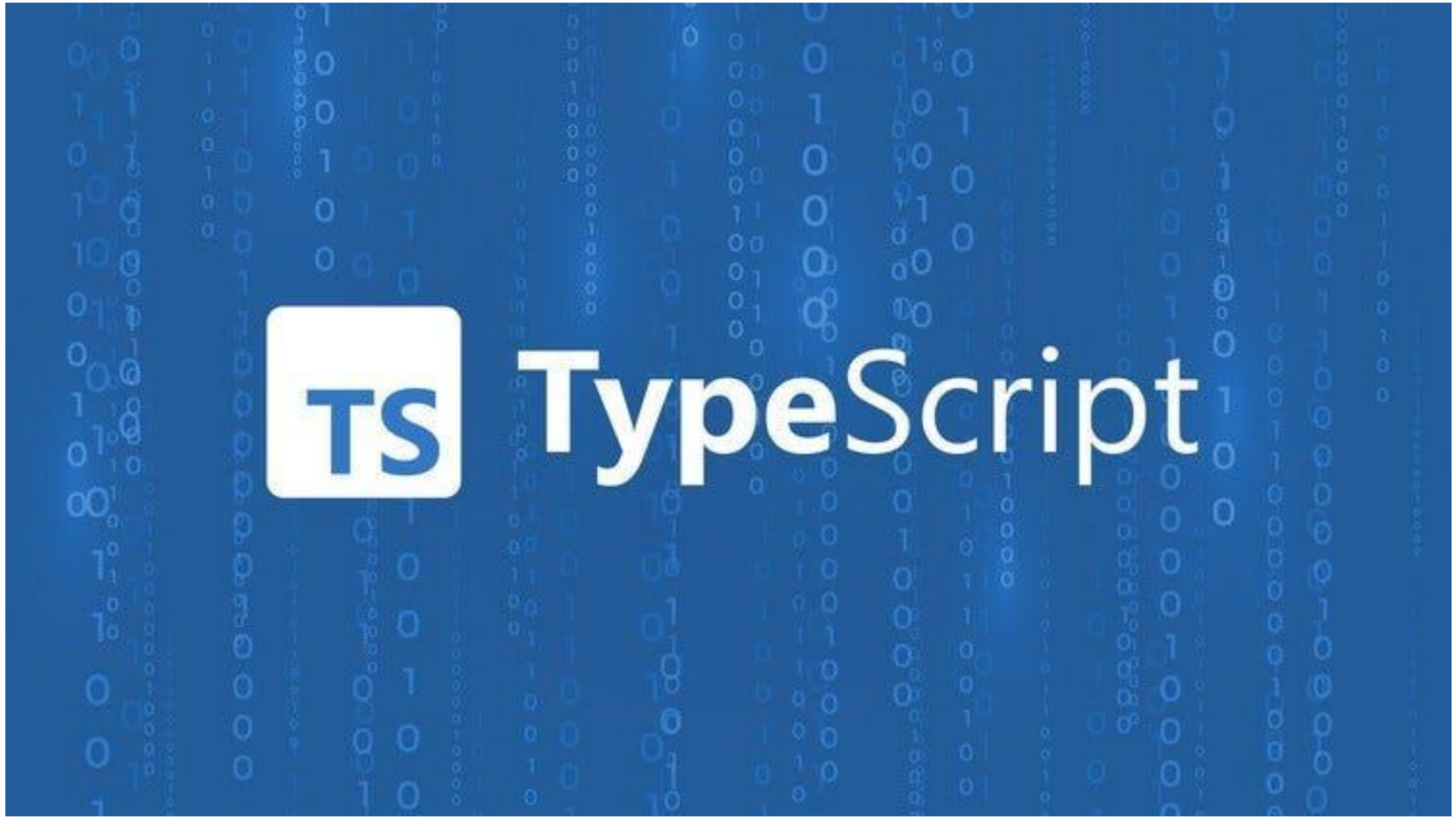
שיעור 26


TS
TypeScript

מה זה Typescript?

Typescript היא שפת תכנות שבנויה על Javascript.
השפה מוסיפה לנו פיצ׳רים חדשים והכי חשוב ביניהם – Strongly typed language

בניגוד לשפות כמו Javascript, Python , Ruby , אנחנו לא מגדירים את סוג המשתנה, משתנה יכול להיות מחרוזת ולאחר מכן מספר ואז מערך.
בשפות כמו Java , C#, C++ אנחנו מחוייבים להגדיר כל משתנה איזה סוג של מבנה נתונים הוא יכיל.

התקנה

הריצו בטרמינל:
npm i –g typescript
למשתמשי מק / לינוקס, ייתכן שתצטרכו להריץ sudo npm i -g typescript

לאחר התקנה הריצו tsc –v ובדקו כי מופיעה גרסא.

Usage

כדי להריץ קבצי TS, נצטרך ליצור קובץ עם הסיומת .ts כדי לסמן שזה קובץ TS בניגוד לJS שאנחנו מסיימים עם .js
מכיוון ודפדפנים לא יודעים לקרוא TS, לשפה יש Compiler משלה שעושה transpile לקוד שנכתוב בTS ובעצם ימיר את זה לJS.
כדי להריץ את הקובץ TS בעזרת הcompiler נריץ בטרמינל tsc <filename>.ts

נוכל לראות שלאחר מכן נוצר קובץ script.js שעבר המרה.

Data Types

JavaScript

- Number
- String
- Boolean
- undefined
- null
- Array
- Object

TypeScript

- any
- unknown
- never
- enum
- tuple

שימוש

דוגמאות ליצירת משתנים בTypeScript:

```
let num:number = 27
let myName:string = "Eitan"
let isAdult:boolean = true
```

שימוש

לאחר שהגדרנו את סוג הערך שיהיה במשתנה, נדע עוד לפני שנריץ את הקוד אם עשינו טעות, לא צריך לחכות להריץ את הקוד ולבדוק איזה בעיות צצו.

```
let num:number = 27
num = "Hello"
```

```
Type 'string' is not assignable to type 'number'.
```

שימוש

מה נחשב כתקלה, מה מבחינתו תקין, מה לא תקין, נוכל לעשות התאמה אישית לפי הצרכים שלנו, מה שנעשה אנחנו נתקין קובץ בשם, לאט לאט עם ההתקדמות בחומר, אנו נכיר עוד ועוד יכולות.

קובץ הגדרה Tsconfig.json :
על מנת להתקין אותו נריץ בטרמינל tsc --init:

Tuple

Tuple הוא מערך באורך קבוע, שלכל אלמנט יש type משלו.
ניתן ליצור Tuple לערכים כמו שם משתמש שמכיל id + name:

```
let user: [number, string] = [1, "Eitan"];
```

לעומת מערך רגיל שכל האלמנטים חייבים להיות מאותו הערך.
שימוש בTuple בעיקר שיש לנו מערכים קטנים (עד 3-2 אלמנטים), במידה ויש לנו מבנה נתונים גדול יותר נשתמש באובייקט / מערך מה שיותר יתאים לנו.

Enum

Enum הוא קיצור של Enumerations (ספירות). לדוגמא קבוצה של Card suits..

נשתמש מתי שנרצה סט של אלמנטים קבועים, לדוגמא אם נרצה סט של מידות בגדים:

במקום לכתוב 3 משתנים שונים:

```
const small = 1;
const medium = 2;
const large = 3;
```

אנחנו ניצור Enum:

```
enum Size {
  Small = 1,
  Medium,
  Large,
}

let mySize: Size = Size.Medium;
```

Functions

גם לפונקציות , נצטרך להגדיר מה הסוג של כל משתנה, וגם מה הערך שיוחזר מהפונקציה, במידה ויוחזר!

```
function calculateVoid(num: number): void {
  // void if no value returns
}

function calculateNumber(num: number): number {
  return 0;
}
```

שימו לב שאנחנו מכריזים על סוג הערך של הפרמטר, ולאחר ה() אנחנו מכריזים איזה ערך יוחזר מהפונקציה. במידה ולא מוחזר כלום, נציין void.

Objects

```
let employeeTs: {
  readonly id: number;
  name: string;
  phone?: number;
  work: (date: Date) => void;
} = {
  id: 1,
  name: "Eitan",
  work: (date: Date) => {
    console.log(date);
  },
};
```

שאנחנו כותבים אובייקט, אנחנו חייבים
להגדיר מראש איזה סוג כל מפתח
באובייקט, זה יכול לגרום לכתיבה מסורבלת
של אובייקטים:
שימו לב למספר ערכים חדשים:
Readonly על הid, זה אומר שלא נוכל
לעולם לערוך את הערך הזה.
?. ערך רשות, לא חובה.
Void – הפונקציה לא מחזירה ערך.

Type

כפי שראינו, כתיבה של כל אובייקט עם הגדרה של כל מפתח באובייקט, יכולה להיות עבודה מסורבלת, לכן אנחנו נשתמש בType על מנת להגדיר ערכים ואז נשתמש בType היכן שנרצה.

```
type Employee = {
    readonly id:number,
    name:string,
    phone?:number
    work?: (date:Date) => void
}

let typeEmployee: Employee = {
    id:2,
    name:"John",
}
```

Type

משימה:

1. צור משתנה ששומר שם פרטי של אדם והדפס אותו ולאחר מכן שנה אותו והדפס שוב. בצע קומפילציה על הקובץ והרץ אותו.
2. צור מערך של מספרים שמכיל מספרים מאחד עד שש.
3. צור פונקציה שמקבלת מערך ומחזירה סכום שלו
4. צור שלוש מכוניות מאותו סוג עם פרמטר לשם , צבע , ושנה והאם היא ליסינג.
5. בונוס 1: הוסף לכל מכונים דגם שהוא אחד מ3 שקיימים בENUM.
6. בונוס 2: צור מערך של 2 מספרים 2 טקסטים ו2 BOOLEANים.

סוף שיעור 26

האם יש שאלות?